Lab06: DLL Hijacking & Control Flow Manipulation

Notepad++ v8.9.3 Installer · NSIS Error String Hijack · POC-1 GOT IT

Report Date	April 2026
Author	Michael.A | SafeTest-Dev
Classification	Confidential – Authorized Use Only
Target Binary	npp.8.9.3.Installer.x64.exe (Notepad++ v8.9.3 NSIS Installer)
Format	PE32+ · x86-64 · Windows Installer · NSIS-based
Analysis Type	Static Analysis + Control Flow Manipulation + String Patching
Tool	Radare2 (r2) · vim · wine
Platform	Arch Linux + Wine
Target Address	0x0040337f — jmp conditional redirect
String Target	0x0040a098 — NSIS Error message in .data section

Findings Overview

ID	Finding	Component	Impact	Severity
ALC-01	Integrity Check Bypassable via Jump Redirect	0x0040337f (je → jmp)	Authentication Bypass	CRITICAL
ALC-02	NSIS Error String Overwritable in .data Section	0x0040a098 (UTF-16LE)	Social Engineering / String Spoof	HIGH
ALC-03	Real-World Installer Vulnerable to Static Patch	npp.8.9.3.Installer.x64.exe	Production Software Compromise	HIGH

> ■ *This documentation is prepared exclusively for defensive security research and educational purposes within the SafeTest-Dev framework. All techniques are documented for defensive understanding and incident response. DO NOT apply these techniques to software outside of a controlled lab environment.*

1. Executive Summary

This report presents the security analysis of **Lab06** — targeting the official Notepad++ v8.9.3 installer (**npp.8.9.3.Installer.x64.exe**) — a real-world NSIS-based PE32+ Windows installer. Unlike previous labs which targeted purpose-built CTF binaries, this lab operates against production software, demonstrating that the same static analysis and binary patching techniques apply to real-world targets.

Two separate manipulation objectives are pursued: Control Flow Manipulation (redirecting a conditional jump to force integrity check bypass at 0x0040337f) and String Manipulation (overwriting the NSIS error message string in .data at 0x0040a098 to display custom output — "POC-1 GOT IT"). Both are confirmed in the NSIS error dialog upon re-execution.

2. Target Overview

2.1 Binary Identification

Field	Value
Binary Name	npp.8.9.3.Installer.x64.exe
Product	Notepad++ v8.9.3 Official Installer
Format	PE32+ executable for MS Windows (GUI)
Architecture	AMD x86-64
Installer Type	NSIS-based (Nullsoft Scriptable Install System)
Linkage	Dynamically linked — KERNEL32.dll, various Windows APIs
Platform	Arch Linux + Wine (cross-platform execution)
Analysis Tool	Radare2 (r2 -A) + vim visual mode

2.2 Manipulation Objectives

#	Objective	Target Address	Technique	Result
1	Control Flow Patch — bypass integrity check	0x0040337f	Redirect je → jmp (2 bytes)	Integrity check bypassed unconditionally
2	String Patch — overwrite NSIS error message	0x0040a098	wx UTF-16LE payload write	Dialog shows "POC-1 GOT IT"

3. Target Identification & Normal Behavior

3.1 Working Directory

The target binary is the official Notepad++ v8.9.3 installer. The working directory contains the installer along with several DLLs bundled with the package.

```
compz@comp1 ~/init>>> ls
NppConverter.dll  NppExport.dll  mimeTools.dll
notepad++.exe     npp.8.9.3.Installer.x64.exe
```

3.2 Normal Execution (Pre-Patch)

Running the installer normally via Wine shows the expected Notepad++ v8.9.3 Setup wizard GUI. Installation completes successfully and the changelog is displayed in the installed Notepad++ instance — confirming the binary is functional and unmodified at this stage.

```
compz@comp1 ~/init>>> wine npp.8.9.3.Installer.x64.exe
```

```
; -> Notepad++ v8.9.3 Setup GUI launches
; -> Installation completes successfully
; -> changelog.log opened in Notepad++ post-install
; -> Notable entry: CVE-2025-14819 fix in v8.9.3
```

Observation: The installer runs cleanly under Wine. This confirms the binary is a standard NSIS installer with an integrity check that passes on a clean copy. Our objective is to manipulate the binary so that (1) the integrity check branch is bypassed and (2) the NSIS error dialog displays our custom string instead of the standard error message.

4. Static Analysis with Radare2

4.1 Opening the Binary

The installer is opened in Radare2 with full automated analysis. The function list reveals a large binary with extensive NSIS runtime functions and Windows API stubs. Key functions of interest are identified by cross-referencing string annotations in the disassembly.

```
compz@comp1 ~/init>>> r2 -A npp.8.9.3.Installer.x64.exe
WARN: Relocs has not been applied.
INFO: Analyze all flags starting with sym. and entry0 (aa)
INFO: Analyze imports (af@@@i)
INFO: Analyze entrypoint (af@ entry0)
INFO: Analyze symbols (af@@@s)
INFO: Analyze all functions arguments/locals (afva@@@F)
INFO: Analyze function calls (aac)

[0x004036f]> afl
0x004036f   83 1565  entry0
0x004011cf  19  170  fcn.004011cf
0x004063c   17  160  fcn.004063c
0x00401434 409 6196  fcn.00401434    <- MAIN NSIS RUNTIME
0x00405761  13  211  fcn.00405761
0x004033d0  23  264  fcn.004033d0
0x00406322  20  378  fcn.00406322
0x004068a   20  175  fcn.004068a     <- integrity check region
...
```

4.2 String Analysis via vim Visual Mode

Navigating to the .data section in vim visual mode within r2 reveals the string table used by the NSIS installer. Key strings are visible in the hex/string dump — including the NSIS integrity check error message that we will later overwrite.

Address	String / Reference	Section	Type	Significance
0x00007efa	GetProcAddress	.rdata	ascii	Windows API import
0x00007f0c	GetModuleHandleA	.rdata	ascii	Windows API import
0x00007f20	GetExitCodeProcess	.rdata	ascii	Windows API import
0x00007f36	WaitForSingleObject	.rdata	ascii	Windows API import
0x00007f4a	KERNEL32.dll	.rdata	ascii	Import library
0x00008020	verifying installer: %d%%	.data	utf16	Install progress string
0x00008054	unpacking data: %d%%	.data	utf16	Unpack progress string
0x0040a098	NSIS Error (integrity check fail)	.data	utf16	← PATCH TARGET (string)

5. Manipulation 1: Control Flow Patch

5.1 Locating the Integrity Check Branch

The NSIS installer performs an integrity check before proceeding with installation. When the check fails (e.g. if the installer file is modified), execution jumps to an error-handling block. By identifying this conditional jump and redirecting it, we can force execution to bypass the error path entirely.

```
; Disassembly around integrity check branch
0x00403356  ff75ec      push dword [dwBytes]
0x00403359  f7d1        not ecx
0x0040335b  83e104      and ecx, 4
0x0040335e  a3080f4200  mov dword [0x420f08], eax
0x00403363  1bc1        sub eax, ecx
0x00403365  8b4df0      mov ecx, dword [var_10h]
0x00403368  56          push esi
0x00403369  53          push ebx
0x0040336a  8d4408e4    lea eax, [eax + ecx - 0x1c]
0x0040336e  6a77        push 0x77
0x00403370  a3fc0e4200  mov dword [0x420efc], eax
0x00403376  e854000000  call fcn.004033d0
0x0040337c  3b45ec      cmp eax, dword [dwBytes]

; ■■■ CONTROL FLOW TARGET ■■■
```

```
; Seek to NSIS error string address
[0x004036f]> s 0x0040a098

; Inspect original string (UTF-16LE)
[0x0040a098]> psu
I\u0000n\u0000s\u0000t\u0000a\u0000l\u0000l\u0000e\u0000r\u0000...
; -> "Installer integrity check has failed. Common causes include
;     incomplete download and damaged media. Contact the installer author..."
```

6.2 Writing the Custom String Payload

The custom string "POC-1 GOT IT" is written as a UTF-16LE encoded byte sequence using the **wx** command. UTF-16LE encodes each ASCII character as 2 bytes with a null byte padding: "P" → 50 00, "O" → 4f 00, "C" → 43 00, etc. The string is null-terminated with a double null (00 00).

```
; Write UTF-16LE encoded "POC-1 GOT IT" + null terminator
[0x0040a098]> wx 50004f0043002d003100200047004f005400490054000000000000000000000000

; Verify final string
[0x0040a098]> psu
P\u0000O\u0000C\u0000-\u00001\u0000 \u0000G\u0000O\u0000T\u0000 \u0000I\u0000T\u0000
; -> "POC-1 GOT IT" <- confirmed
```

String Patch Applied: Address 0x0040a098 — Original: "Installer integrity check has failed..." — Patched: "POC-1 GOT IT" — Encoding: UTF-16LE (2 bytes per char + double null terminator). When the NSIS error dialog is triggered, it now displays "POC-1 GOT IT" instead of the standard NSIS error message.

6.3 UTF-16LE Encoding Reference

Char	Hex (UTF-16LE)	Char	Hex (UTF-16LE)	Char	Hex (UTF-16LE)
P	50 00	O	4f 00	C	43 00
-	2d 00	1	31 00		20 00
G	47 00	O	4f 00	T	54 00
	20 00	I	49 00	T	54 00
\0	00 00	(end)			

7. Verification & Proof of Concept

7.1 Re-execution After Patches

The patched binary is executed via Wine. With both patches applied — the control flow redirect at 0x0040337f and the string overwrite at 0x0040a098 — the NSIS error dialog appears with the custom string "POC-1 GOT IT" instead of the standard integrity check failure message.

```
compz@comp1 ~/init>>> wine npp.8.9.3.Installer.x64.exe
; -> NSIS installer launches
; -> Integrity check branch triggered (patched binary)
; -> Error dialog appears with custom string:

+----------------------------------+
| NSIS Error                   [X] |
|                                  |
|   [!]  POC-1 GOT IT              |
|                                  |
|           [ OK ]                 |
+----------------------------------+

; POC confirmed: custom string displayed in NSIS dialog
```

7.2 Technical Patch Summary

Parameter	Manipulation 1 (Control Flow)	Manipulation 2 (String Patch)
Target Address	0x0040337f	0x0040a098
Section	.text (code section)	.data (data section)
Original	je 0x00403388 (conditional jump)	"Installer integrity check has failed..."
Patched	jmp 0x004030fc (unconditional jump)	"POC-1 GOT IT" (UTF-16LE)
Bytes Modified	2 bytes (eb 7b)	26 bytes (UTF-16LE payload + null term)
Tool Used	oo+ / wx eb7b	oo+ / wx 50004f00430...
Effect	Integrity check bypassed completely	Custom string shown in NSIS error dialog

8. Vulnerability Findings

ALC-01	CRITICAL
Component	0x0040337f — NSIS integrity check conditional jump (.text)
Method	Static analysis (r2 pdf) + 2-byte jump redirect (wx eb7b)
Impact	Integrity check bypassed unconditionally — tampered binary runs without error

The NSIS installer implements a CRC-based integrity check to verify that the installer binary has not been tampered with. However, the check result feeds into a single conditional jump (je 0x00403388) at 0x0040337f. Redirecting this jump to an alternate address bypasses the entire error path unconditionally. Any attacker who can write 2 bytes to the binary can completely bypass the integrity protection — making the check ineffective against targeted tampering.

ALC-02	HIGH
Component	0x0040a098 — NSIS error string in writable .data section (UTF-16LE)
Method	Static address targeting + wx UTF-16LE payload write in r2
Impact	Arbitrary string shown in NSIS error dialog — social engineering vector

The NSIS error message is stored as a UTF-16LE string in the writable .data section at a known static address (0x0040a098). Because the address is fixed (no ASLR on this binary) and the section is writable via r2 in write mode, the string can be overwritten with arbitrary content. This technique can be leveraged for social engineering — displaying misleading dialogs to users who run a tampered installer.

ALC-03	HIGH
Component	npp.8.9.3.Installer.x64.exe — real-world production binary
Method	Standard static analysis + binary patching workflow (same as Lab04/Lab05)
Impact	Production software as vulnerable to static binary patching as CTF binaries

Unlike previous labs which used purpose-built CTF binaries, Lab06 demonstrates that the same static analysis and binary patching workflow applies directly to production software. The Notepad++ installer provides no additional protection against static binary modification beyond the integrity check itself — which is bypassed by the control flow patch.

9. Risk Assessment

ID	Finding	Likelihood	Impact	Overall
ALC-01	Integrity Check Bypassable via Jump Redirect	High	Critical	CRITICAL
ALC-02	NSIS Error String Overwritable in .data Section	High	High	HIGH
ALC-03	Real-World Installer Vulnerable to Static Patch	High	High	HIGH

10. Recommendations

#	Recommendation	Impact
1	Implement multi-point integrity checks — not just one CRC-based jump	Eliminates single-point bypass
2	Use digital code signing + verify signature at runtime before execution	Detects any binary modification
3	Enable PIE/ASLR on the installer binary to randomize all addresses	Prevents static address targeting
4	Store UI strings in read-only .rdata section — not writable .data	Prevents string overwrite attacks
5	Implement self-integrity verification (hash own .text section at runtime)	Detects code section tampering
6	Obfuscate jump targets around integrity check logic	Raises analysis bar for attackers

11. Conclusion

Lab06 successfully demonstrates two independent binary manipulation techniques against a real-world production installer. Control flow manipulation bypassed the NSIS integrity check by redirecting a single 2-byte conditional jump. String manipulation overwrote the UTF-16LE error message string in the .data section with a custom payload ("POC-1 GOT IT"), which was confirmed in the NSIS error dialog upon re-execution.

The lab reinforces a core principle from the series: binary protections that rely on a single check point — whether a key comparison, an XOR-encoded constant, or an integrity check jump — are fundamentally fragile against an analyst with static analysis tools. Two bytes and one hex write were sufficient to compromise both protection mechanisms in a production binary.

- ALC-01 (Critical) — Single conditional jump at 0x0040337f bypassed with 2-byte redirect — integrity check rendered ineffective.
- ALC-02 (High) — UTF-16LE NSIS error string at 0x0040a098 overwritten with "POC-1 GOT IT" — dialog confirmed.
- ALC-03 (High) — Production installer (Notepad++ v8.9.3) offers no additional resistance to static binary patching.

Lab Progression: Lab01 required runtime patching. Lab02 needed XOR inversion. Lab03 required modular arithmetic. Lab04 introduced binary patching. Lab05 introduced multi-stage analysis. Lab06 applies all techniques to real-world production software — confirming that the methodology is target-agnostic.

12. Appendix — Tools & References

Tool / Concept	Purpose
file	Binary format identification (PE32+, NSIS installer)
wine	Execute Windows installer binary on Arch Linux
r2 / Radare2	Static analysis framework — disassembly + binary patching
r2: afl	Function listing — NSIS runtime + integrity check functions

Tool / Concept	Purpose
r2: pdf @ addr	Full function disassembly
r2: vim / V	Visual hex/string browser for .data section inspection
r2: psu	Print UTF-16LE string at current address
r2: oo+	Reopen binary in read-write mode
r2: wx / wa	Write hex bytes / write assembly at target address
je → jmp (eb7b)	Control flow patch — 2-byte unconditional jump replacing conditional
UTF-16LE encoding	NSIS wide string format — 2 bytes per ASCII char + double null
0x0040337f	Integrity check jump — patch target (control flow)
0x0040a098	NSIS error string — patch target (string overwrite)
POC-1 GOT IT	Custom payload displayed in NSIS error dialog — proof of concept

End of Report · SafeTest-Dev | Binary Security Research | Michael.A | April 2026

LAB06 — COMPLETED · Control Flow: 0x0040337f BYPASSED · String: "POC-1 GOT IT" CONFIRMED